

Kristálytiszta Elektronika 2

A Kristálytiszta Elektronika tananyag fejlesztését az Európai Bizottság támogatta az Erasmus+ program keretén belül „2020-1-HU01-KA201-078779” számú „Elektronika és programozás témájú innovatív tananyag fejlesztése a köznevelés számára” című projekt kapcsán.

Az Európai Unió
Erasmus+ programjának
társfinanszírozásával



A projektet az alábbi 4 intézményből álló nemzetközi partnerség valósította meg:

- Xtalin Mérnöki Tervező Kft. – Budapest
- ELTE Bolyai János Gyakorló Általános Iskola és Gimnázium – Szombathely
- Bolyai Farkas Elméleti Líceum – Târgu Mureș
- Pro Ratio Alapítvánnyal karöltve működő Madách Imre Gimnázium – Šamorín



XTALIN



Szerzői jogok:

Jelen tananyag az Xtalin Mérnöki Tervező Kft., mint koordinátor által vezetett partnerség szellemi terméke. Az anyagok oktatási felhasználásra készültek, ezért e célra szabadon felhasználhatóak; tartalmuk azonban nem módosítható, továbbfejleszthető az Xtalin Kft. írásban adott engedélye nélkül. Az anyagok újraközlése tartalmilag változatlan formában, kizárólag a tananyag szerzőinek és az eredeti tananyag elérhetőségének világos megjelölésével lehetséges, kizárólag az Xtalin Kft. írásban adott engedélyével.

Kapcsolat: <http://kristalytisztaelektronika2.hu/>
info@kristalytisztaelektronika.hu

12. D-A átalakítás

Írta: Vincze Viktor és Baracsi Gábor

Lektorálta: Preinsperger Máté

Az analóg-digitális átalakítást már részletesen megismertük az előző, illetve a Kristálytisza Elektronika 22. tananyagrészében. Az analóg jel mérése és digitális jellé alakítása szükséges ahhoz, hogy a jel tárolható vagy feldolgozható legyen digitális eszközeinkkel. Azonban a világ nem digitális, ahhoz, hogy ezek a jelek újra emberek, valamint a külvilág számára érzékelhetővé váljanak, digitális-analóg átalakításra van szükség. Tehát a digitális-analóg átalakítás a kapocs a digitális világ és a külvilág között, mely már a processzorok megjelenése óta jelen van, de csak az elmúlt két-három évtizedben, a monolitikus, tehát szilícium egykristályra integrált DAC-k (ejtsd: déacé, Digitális-Analóg Átalakítók, Digital-Analog Converters) megjelenésével vált széles körben elérhetővé.

A DAC-K ALKALMAZÁSI LEHETŐSÉGEI

A DAC-k számos alkalmazási lehetőséggel rendelkeznek, némelyikben első ránézésre nyilvánvaló a digitális-analóg átalakítás funkciója, máshol mélyebbre kell ásunk, hogy felfedezzük a szerepét.

Az egyik legkézenfekvőbb példa, amivel már mindenki találkozott, az a zene. Miután a stúdióban elkészülnek a hangfelvételek, azokat a hangmérnökök megszerkesztik, majd a digitális kópiák készen is állnak a terjesztésre. Azonban ez így nem hallható, hiszen a hang digitális formátumban van tárolva, de ahhoz, hogy fogyasztható legyen, a digitális reprezentációt vissza kell alakítani hanggá.

A következő ilyen példa a videó, amely sok hasonlóságot mutat az előző ponttal. A digitális formában tárolt vagy streamelt filmek megjelenítéséhez szükség van a videojelek analóggá alakítására. Például VGA csatlakozó használata esetén már a számítógépben megtörténik az digitális-analóg átalakítás. A modern monitor-csatlakozások már általában digitálisak, erre példa a HDMI, aminek a használata esetén a monitoron belül történik meg a pixelek meghajtására szolgáló piros, zöld, kék "jelek" előállítása.

Egyes felügyeleti rendszerekben az ADC digitalizálja a mérendő analóg jeleket, majd ezeket továbbítja feldolgozásra a mikrokontroller felé. Ezek után a vezérlő elküldi az egységnek a mért érték alapján számított beavatkozó jelet, amely visszaalakítja azt analóg jellé.

A legtöbb digitális potméter is DAC-re épül. Ezek a potméterek rendelkeznek valamilyen digitális interfésszel, általában I²C-vel. A DAC architektúráról a tananyagrészben később majd részletesen lesz szó.

A következő alkalmazási példa a motorvezérlés. A 17. tananyagrészben részletesen szó lesz a különböző motortípusokról és azok vezérléséről, ezért most csak a DAC felől közelítve érintjük a témát. Az egyenáramú (DC, Direct Current) motorok esetében például a DAC segítségével is előállíthatjuk a motor vezérlésére szolgáló analóg jelet. A fejlesztőkártyán használt BD6211 nevű motorvezérlő támogatja a VREF lábán az analóg jellel történő vezérlést. Bár vannak olyan motoros alkalmazások,

amelyek nem igényelnek szabályozást, azonban a legtöbb feladathoz ez elengedhetetlen. Ez általában egy mikrokontrollerből, egy DAC-ból, egy motorvezérlőből és az enkóder által mért jelekből áll. Ez a téma részletesen szerepel majd a 18. tananyagrészen. A DAC szerepe a motorvezérlésben az, hogy a mikrokontrollertől kapott digitális adatot analóg feszültség- vagy áramjellé alakítja a motorvezérlő számára.

Számos feladatban, például tesztberendezésekben, szenzorokban, különféle automatizált gyártó és gyártástámogató berendezésekben szükség van a mérő-, vagy beavatkozási eszközök kalibrálására. A kalibrálás egyik módja az, hogy a mérnökök a munkájuk során ellenőrzik és beállítják a szükséges paramétereket. Azonban az ipari automatizálás elterjedésével megjelent az igény, hogy a különféle rendszerek hibái dinamikusan kezelhetők legyenek a hibajelek automatikus mérésével és változtatható kalibráló jelek segítségével. A kalibráló jelek előállítása DAC-kel történik. Olyan paraméterek egyszerű dinamikus állítására van így lehetőség, mint például nullponti feszültséghiba, amiről a DAC főbb jellemzői között is szó lesz. Természetesen még számos egyéb területen alkalmaznak DAC-t, itt csak néhány tipikus példa került bemutatásra.

A DAC FŐBB JELLEMZŐI

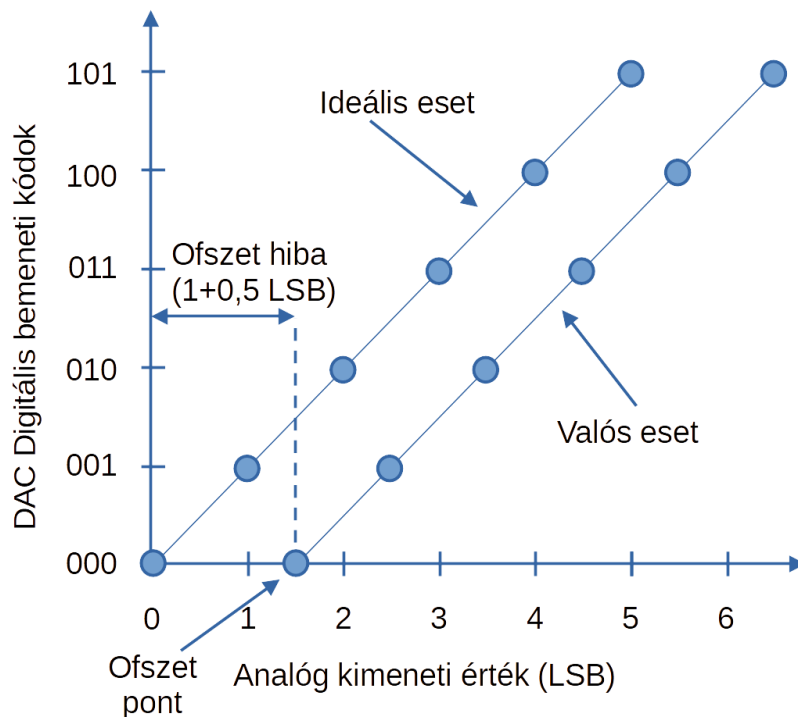
A DAC-k főbb jellemzőiről lesz szó ebben a bevezetésben. A pontos fizikai megvalósításról a tananyag rész későbbi részében olvashatunk, azonban az általános felépítés ismeretére már korábban szükség lesz, ezért a továbbiakban ezt mutatom be. A DAC-k a digitális bemeneti értéket alakítják át analóg jellé. Az analóg kimenetnek követni kell a bemenet változását, amit tipikusan kapcsolható feszültség- vagy áramforrások összegzésével oldanak meg.

Kimeneti tartomány – A DAC legnagyobb és legkisebb kimeneti jelének feszültség- vagy áramerősségbeli különbsége.

Felbontás – Alapegysége a bit. A gyakorlatban megfogalmazva azt jelenti, hogy 1 LSB-nyi bemeneti változás esetén mekkora értékkel (feszültség vagy áram) változik a DAC kimenete. Például egy 8 bites DAC kimeneti tartománya 255 részből áll (2^8-1), így a felbontás a **kimeneti tartomány** 255-öd része.

Maximális mintavételi sebesség - A legnagyobb sebesség, amivel a DAC még a specifikációnak megfelelő kimenetet képes előállítani.

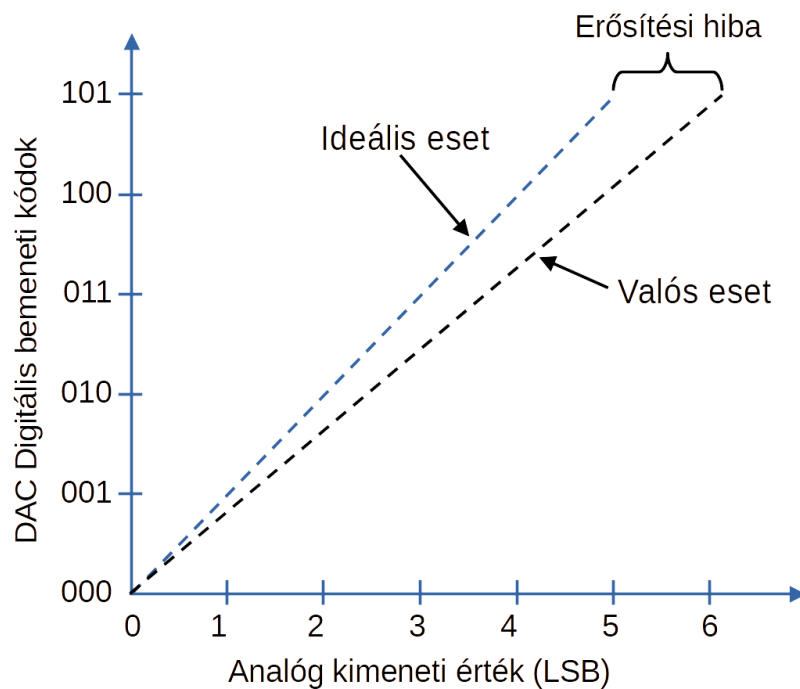
Nullponthiba - Az ideális és a valós kimeneti érték különbsége, amikor a digitális bemeneti érték nulla.



1. ábra: DAC nullponthibája

A fenti ábra szemlélteti a nullponti (ofszet) hibát. Jól látható, hogy az ideális egyenestől a valós kimeneti jel a 000 bemeneti kód esetén 1,5 LSB értékkel eltér. A hiba legegyszerűbb kompenzálásához meg kell mérni a nulla digitális bemenethez tartozó kimeneti értéket, majd ezt visszaszámolni digitális értékkel és levonni a további bemeneti kódokból.

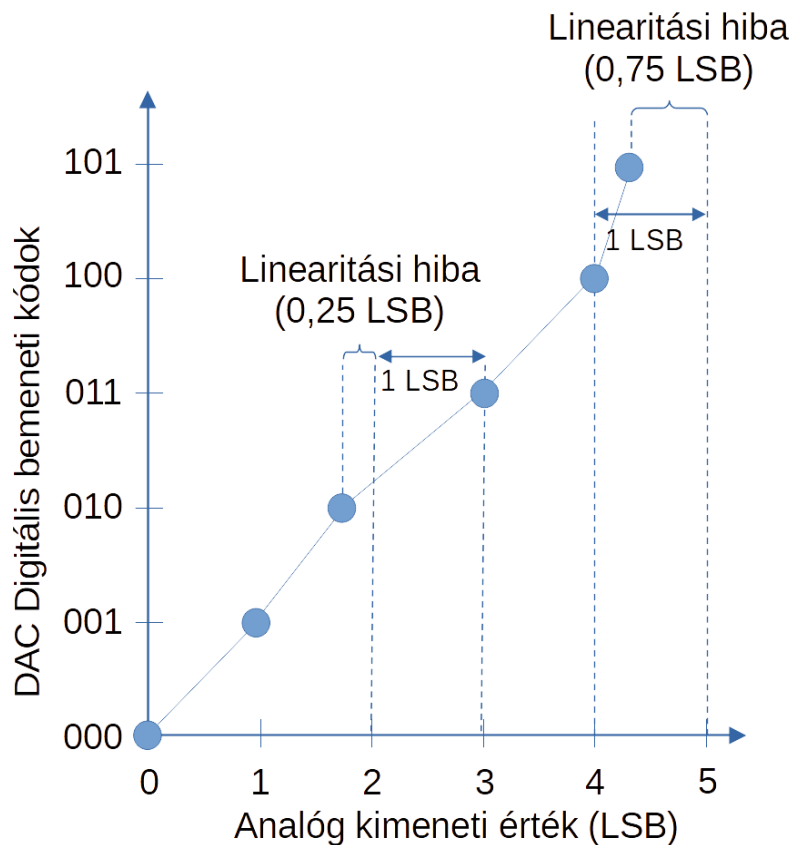
Erősítési hiba - Az ideális és a valós kimeneti érték különbsége a nullponti hiba korrigálása után, maximális digitális bemeneti érték esetén.



2. ábra: DAC erősítési hiba

A fenti ábra szemlélteti az erősítési hibát. Mivel a két egyenes meredeksége különbözik, látható, hogy maximális bemenet esetén lesz a legnagyobb különbség az ideális és a valós kimenet között.

Differenciális linearitási hiba (DNL: Differential nonlinearity) - Ideális esetben, ha a digitális bemeneti értéket eggyel növeljük, akkor a kimenetnek 1 LSB értékkel kell növekedni. Az 1 LSB-től való eltérés a DNL. Ha a DNL értéke több, mint -1 LSB, akkor akár az is előfordulhat, hogy az eszköz nem monoton és a bemenet növelése hatására a kimenet csökken. Ezt nevezzük **monotonitási hibának**.

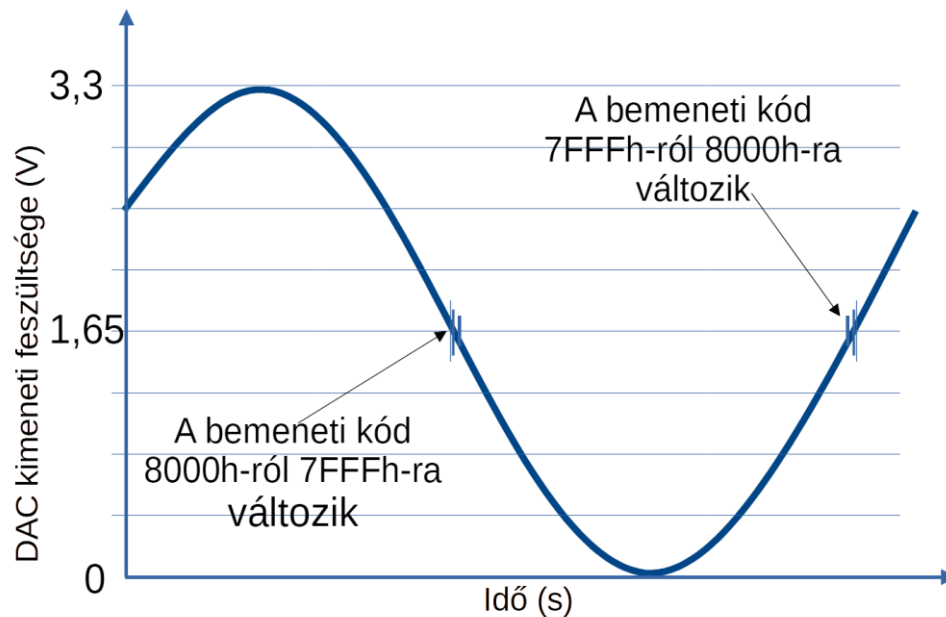


3. ábra: Differenciális linearitási hiba

A fenti ábra szemlélteti a linearitási hibát, látható, hogy a digitális bemenet (y tengely) függvényében az analóg kimenet (x tengely) nem változik arányosan. Egységnyi bemeneti változáshoz nem 1 LSB kimeneti jelváltozás tartozik.

Beállási idő - Megadja, hogy mennyi ideig tart a kimeneten az állandósult állapotbeli érték (± 1 LSB hibán belüli) beállása, a digitális bemenet minimum értékről maximum értékre váltása esetén.

Kapcsolási tranziens - A jelenség során a kimeneti jelen nemkívánatos tüskék jelennek meg, amiket a DAC kapcsolóinak nem egyszerre kapcsolása okoz. Ezekről a kapcsolókról a DAC típusok részben még bővebben fogtok olvasni.



4. ábra: DAC kapcsolási tranziens hibajelenség

A fenti ábra szemlélteti a kapcsolási tranzienszt. Annak ellenére, hogy a bemenetre ideális digitális szinuszjelet kapcsolunk, a kimeneten mért analóg szinuszjelen megjelennek a nem kívánatos tranziensek akkor, amikor az egyes helyiértékek változnak.

Jel-zaj viszony (SNR: Signal to noise ratio) - A kimeneten lévő hasznos jelnek és a zajnak az aránya. Mivel a jel és a zaj különböző fizikai értelemben vett nagysága általánosságban több nagyságrenddel eltér, ezért ennek az aránynak a meghatározása logaritmikus skálán történik, mértékegysége a decibel. A hasznos jel és a zaj teljesítményének (P) mérésével a következő képlet alapján határozzák meg:

$$\text{SNR}_{\text{dB}} = 10 \cdot \log_{10} \left(\frac{P_{\text{jel}}}{P_{\text{zaj}}} \right)$$

Ha viszont a jel és a zaj amplitúdójára szeretnénk elvégezni a számítást, akkor a következő egyenletet kell használnunk:

$$\text{SNR}_{\text{dB}} = 20 \cdot \log_{10} \left(\frac{A_{\text{jel}}}{A_{\text{zaj}}} \right)$$

A korábban ismertetett hibák közül az alkalmazás dönti el, hogy melyekben lehetséges kompromisszumot kötni. Az audio alkalmazásokban használt DAC esetében fontos a nagy felbontás (általánosan elterjedt a 24 bites felbontás) és a minimális torzítás, a mintavételi frekvencia pedig alacsony, általában maximum 192 kHz. Míg a videókhoz használt DAC-k a sebességet helyezik előtérbe, például a mobiltelefonokban használtak több GHz mintavételi frekvenciával és akár 16 bites felbontással is rendelkezhetnek. Egy mikrokontrollerbe integrált DAC számos feladatra alkalmas és egyszerűsíti is a tervezési folyamatot, hogy nem szükséges diszkrét DAC alkalmazása. Az egyes elektronikai alkatrészek esetén beszélhetünk diszkrét, azaz külön áramköri elemeket igénylő, vagy integrált, tehát egy áramköri elemen belül helyet kapó megvalósításokról. Tipikus alkalmazása az integrált DAC-nek kisebb motorok vezérlésének, szabályozásának megvalósítása, szenzorok kalibráló jelének előállítása, vagy kompromisszumokkal ugyan, de akár audio fájlok lejátszása.

DAC-K TÍPUSAI

Az alkalmazás függvényében számos DAC típus közül választhatunk. Nincs olyan, ami minden tekintetben a legjobb lenne, mert vannak paraméterek, amik optimalizálása egymással ellentétes. Erre jó példa a felbontás és a maximális mintavételi sebesség, ezért mindig a feladat ismeretében érdemes DAC-t választani. A következőkben a teljesség igénye nélkül, a legegyszerűbb típusoktól építkezve haladunk a komplex megoldások felé.

Váltókapcsoló - A legegyszerűbb példa, amire először nem is biztos, hogy DAC-ként gondolnánk, az egy kapcsoló, aminek a GND és egy referencia feszültség között lehet a kimenetét váltani. Ez egy 1 bites DAC és nagyon egyszerű integrált áramkörön megvalósítani.

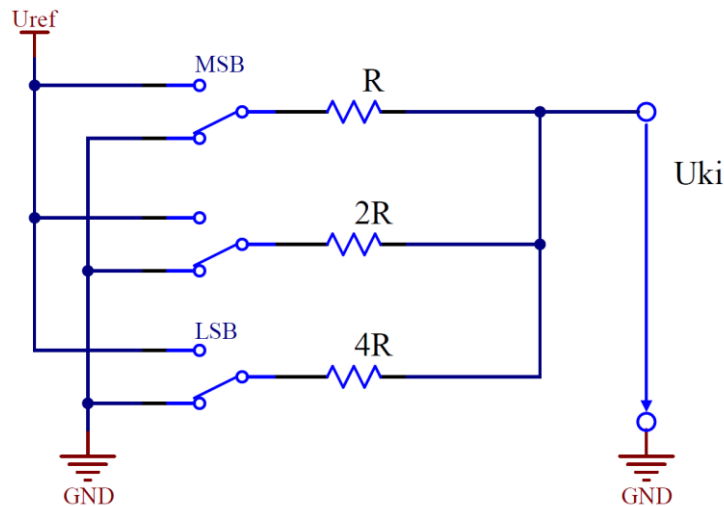
Integráló típusú DAC-k - Megvalósítása impulzusszélesség-modulációval (PWM: Pulse-width modulation) vagy pulzussűrűség-modulációval (PDM: Pulse density modulation) és analóg szűréssel történik.

PWM alapú DAC - Az előző pontban ismertetett kapcsoló két állásához tartozó időtartam arányát fix időszeléken belül a bemeneti digitális jel határozza meg. További elem egy aluláteresztő szűrő, amellyel lehetségessé válik a kimeneti feszültség vagy áram "átlagolása". Számos esetben az aluláteresztő szűrő a szabályozandó eszköz maga, például egy motor, vagy egy LED lámpa esetében az emberi szem. A PWM kimenet a legtöbb mikrovezérlőben elérhető hardveres támogatással, de korlátozott módon szinte bármilyen mikrokontrollerrel megvalósítható, valamely digitális kimenet szoftveres állítása által.

Túlmintavételezésen alapuló DAC (Sigma-delta DAC) - Nagyon hasonló a működési elv, mint a PWM alapú DAC esetében, csak ez a megoldás PDM-et használ, tehát a változó szélességű impulzusok helyett fix frekvenciával vagy 0-ra, vagy 1-re állítja a kimenetet. A sigma-delta DAC előnye, hogy nagy felbontást és nagyon jó linearitást lehet vele alacsony előállítási költséggel elérni. Ennek köszönhetően az audio DAC-k általában ezen a technológián alapulnak.

Binárisan súlyozott DAC-k - Az ebbe a kategóriába tartozó DAC-k esetében minden kettes számrendszerbeli helyiértékkel arányos jel előállításához külön elektronikai alkatrészek és kapcsolók tartoznak. Ezek a DAC-k magas frissítési sebességgel rendelkeznek, nagy pontosságú változatok gyártása azonban nem egyszerű feladat, mivel az integrált áramkörökben nehéz a szükséges arányos fokozatok előállítása. Attól függően, hogy az arányos jelet hogyan állítják elő, számos DAC-t sorolunk ebbe a családba, például:

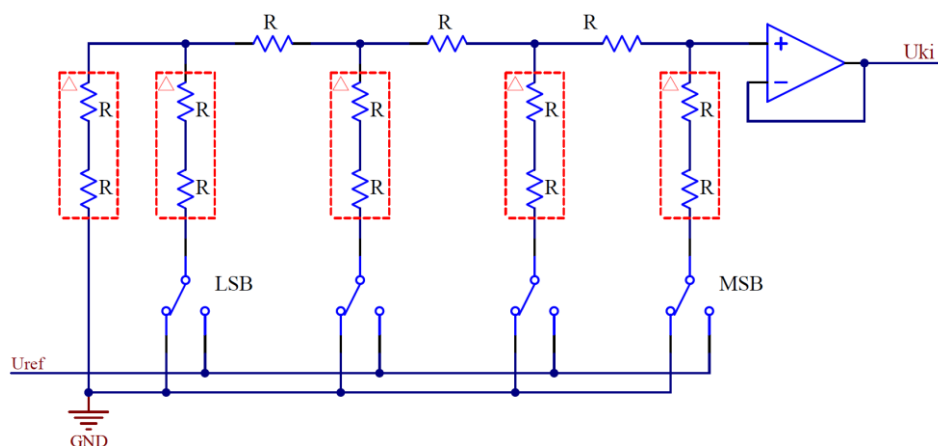
Kapcsolt ellenállásos DAC - Az egyes helyiértékekhez tartozó ellenállásokat vagy föld, vagy referencia potenciálra tudjuk kapcsolni a digitális bemeneti értéktől függően, így a párhuzamos kapcsolt alkatrészek által kapott eredő ellenállások határozzák meg a kimeneti feszültséget. A következő ábrán egy 3 bites DAC példakapcsolás látható.



5. ábra: Kapcsolt ellenállásos DAC

Kapcsolt kapacitásos DAC - A megvalósítása megegyezik a kapcsolt ellenállásos DAC-vel, csak az ellenállások helyett kondenzátorokat alkalmazunk.

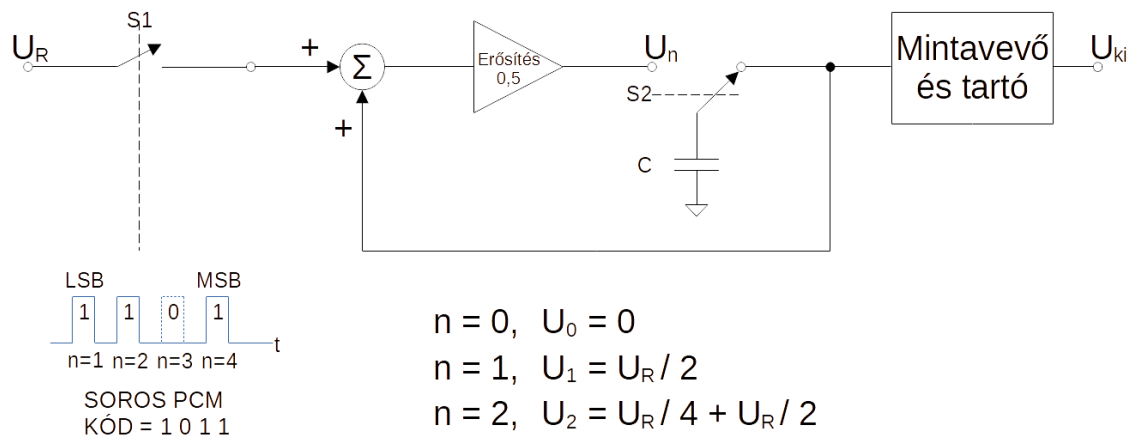
R - 2R létrahálózatos DAC - A kapcsolt ellenállásos DAC-hoz képest ennél a megoldásnál R és $2R$ a felhasznált ellenállások értéke, amelyek közül a $2R$ értékű alkatrész az integrált áramkörökön általában 2 darab sorba kötött R nagyságú ellenállással (ezek a piros keretekben láthatóak) van megvalósítva. Ennek köszönhetően a DAC sokkal könnyebben gyártható és érzékenyebb az R ellenállás értékének pontosságára. A kapcsolók segítségével tudjuk az ellenállásokat föld- vagy referenciapotenciálra kapcsolni. A műveleti erősítőre jutó feszültségérték az egyes ellenállások soros és párhuzamos kapcsolásából adódik. Például, ha csak az MSB nevű kapcsolót kapcsoltuk a referenciapotenciálra, a többi pedig földpotenciálra, akkor az MSB kapcsolóhoz tartozó ellenálláson kívüliek eredője $2R$, így a műveleti erősítő pozitív lábára jutó feszültség pontosan a referencia feszültség fele.



6. ábra: R - 2R létrahálózatos DAC

Ciklikus soros DAC (Cyclic Serial DAC) - A ciklusos DAC-nál az eredményt nem egy lépésben kapjuk meg, hanem egyesével haladunk végig a bináris bemeneti érték helyiértékein és az azokhoz tartozó kimeneteket határozzuk meg. A folyamat a legkisebb helyiértéken kezdődik, amely alapján egy kapcsolóval vagy referencia feszültséget vagy nullát kapcsolunk egy 0,5-szörös erősítésű erősítő

összegző bemenetére, tehát az összegző lépések száma megegyezik a helyiértékek számával. Az ábrán látható a DAC egyszerűsített felépítése, valamint az átalakítás lépései egy 4 bites bemeneti kód esetén az egyes órajelciklusokhoz tartozó kimeneti értékekkel.



$$n = 0, U_0 = 0$$

$$n = 1, U_1 = U_R / 2$$

$$n = 2, U_2 = U_R / 4 + U_R / 2$$

$$n = 3, U_3 = U_R / 8 + U_R / 4$$

$$n = 4, U_4 = U_R / 16 + U_R / 8 + U_R / 2 = 11 U_R / 16$$

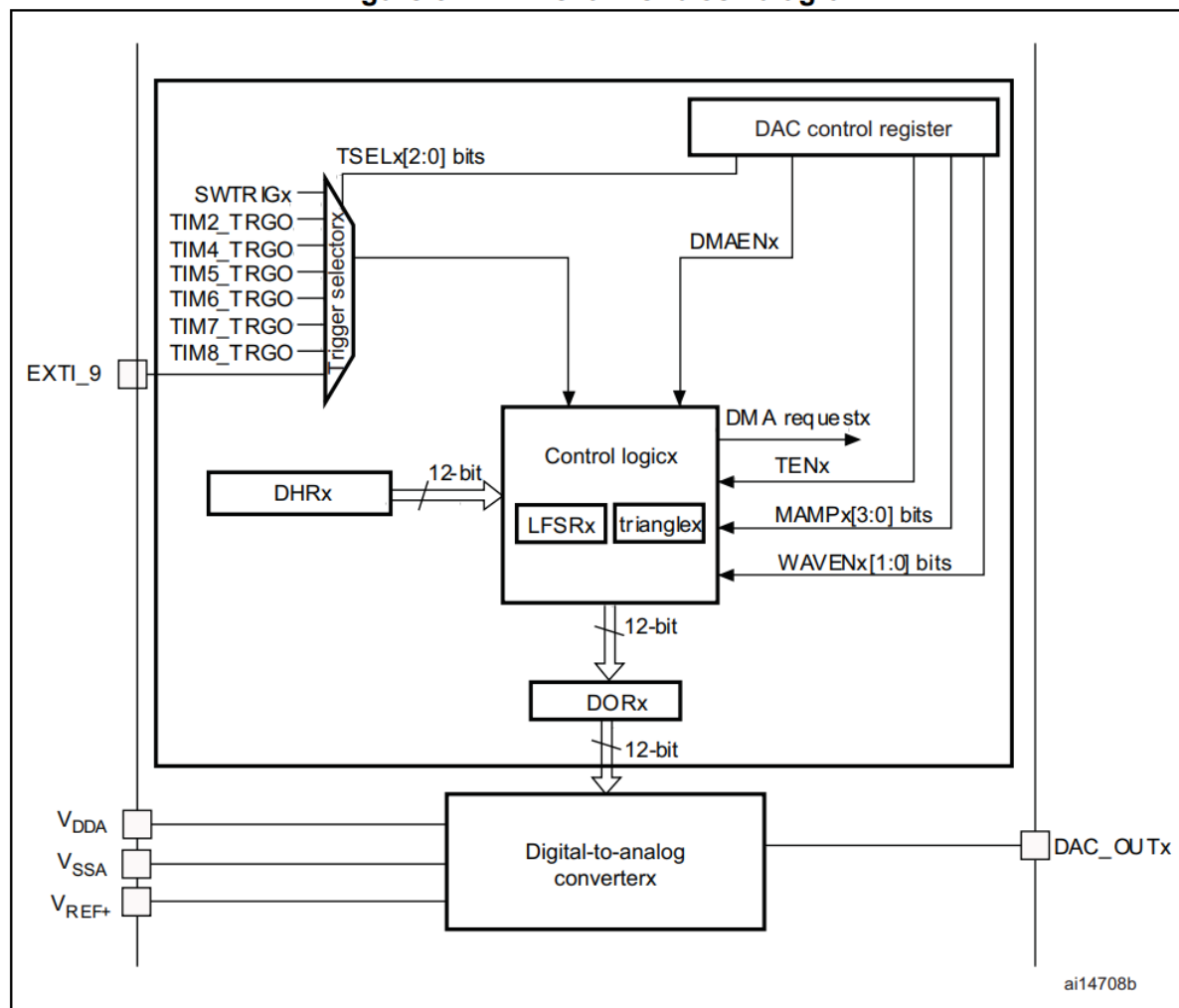
7. ábra: Ciklikus soros DAC

STM32F4 DAC ISMERTETŐ

A fejlesztőkörnyezetben lévő STM32F4 mikrovezérlőben kettő független, feszültségkimenetű R-2R DAC található. A konverzió a beállítástól függően 8 vagy 12 bites felbontásban, a két csatornán egymástól függetlenül vagy szinkron módban is történhet. A kimenetek külön kalibrálhatóak és lehetőség van beépített háromszög- és zajjel kapcsolására a DAC bemenetére. A DAC pineket más perifériák is használhatják, tehát ha a DAC funkciót szeretnénk használni, akkor ennek megfelelő konfiguráció szükséges. Az átalakító használható közvetlen memória-hozzáféréssel (DMA - Direct memory access) üzemmódban, amelynek köszönhetően a mikrokontroller CPU-ja tehermentesíthető.

A következő ábrán a DAC egyszerűsített blokkvázlata látható az adatlapból. Ez némi magyarázatot igényel, de segítségével könnyebben megérthetjük a periféria funkcióit.

Figure 64. DAC channel block diagram



8. ábra: DAC csatorna blokkdiagramja

Forrás: STMicroelectronics

https://www.st.com/resource/en/reference_manual/dm00031020-stm32f405-415-stm32f407-417-stm32f427-437-and-stm32f429-439-advanced-arm-based-32-bit-mcus-stmicroelectronics.pdf

A téglalapok a regisztereket és logikákat jelentik, a nyilak pedig az adatáramlás irányát. A szögletes zárójelben lévő számok, például a [3:0] azt mutatják meg, hogy a regiszter melyik bítjéről van szó. Az elnevezések végén az x azt jelenti, hogy az adott bitek mindkét csatornához külön megtalálhatóak a mikrovezérlőben, csak az ábrán az egyszerűség kedvéért egyszer szerepelnek.

A blokkdiagramon látható egyes komponensek a következők:

DAC vezérlőregiszter (DAC control register) - A csatornák vezérlése a DAC vezérlő regiszterén (DAC control register) keresztül történik. A 32 bites regiszter alsó 16 bitje az egyes csatornához, a felső pedig a kettőshöz tartozik. Ebben lehetőség van a csatorna, a DMA, a trigger, valamint a beépített jelgenerátor egyes funkcióinak engedélyezésére is.

Indítási feltételválasztó (Trigger selector) - A vezérlőregiszterben beállított triggerfeltételt továbbítja a vezérlőlogika felé. A konverziót a számláló, a külső jel és a szoftver is indíthatja. Ha nem használjuk ezeket a triggerelési lehetőségeket, akkor a DAC kimenetei minden órajelciklus után frissülnek, viszont bármilyen triggerelési opció esetén ez csak 4 órajelciklusonként fog bekövetkezni.

Adattároló regiszter (DHR, Data hold register,) - Ebbe a regiszterbe kell beírni a DAC átalakítás digitális bemeneti értékét. A beállítástól függően 8 vagy 12 bites, balra vagy jobbra rendezett számot fogad.

Kimeneti regiszter (DOR, Data output register) - Közvetlenül nem írható, kezelését a vezérlőlogika végzi.

Vezérlőlogika (Control logic) - A vezérlőregiszterbe írt konfiguráció alapján végzi a DAC vezérlését. A DMA kérések és a megszakítások kezelése, az adatok DHR-ből a DOR-be áttöltése is feladata. A beépített jelgenerátor jelét is a vezérlőlogika állítja elő.

Digitális-analóg átalakító (Digital to analog converter) - Miután megismerkedtünk a vezérlést biztosító logikákkal, térjünk is rá magára a DAC-re, aminek a fizikai megvalósítása ebben a blokkban történik. A DAC kimenetei egymástól függetlenül vagy egyidejűleg is frissíthetők. A pontosság további növelésének érdekében lehetőség van külső referenciafeszültség használatára is a V_{REF+} lábon keresztül.

A DAC digitális bemeneti értéke alapján az analóg kimenete a következő képlettel számolható:

$$U_{\text{kimenet}} = \frac{\text{Digitális bemeneti érték(DHR regiszter)} \cdot V_{REF}}{2^n - 1}$$

A képletben az n hatványkitevő jelenti a bitszámot. Ez alapján ki tudjuk számolni, hogy 1 V analóg kimeneti feszültséghez mekkora értéket kell a bemenetre kapcsolnunk. 12 bites felbontás és 3,3 V-os referenciafeszültség mellett a képlet a következőképpen alakul:

$$\text{Digitális bemeneti érték(DHR regiszter)} = \frac{U_{\text{kimenet}} \cdot (2^n - 1)}{V_{REF}} = \frac{1 \text{ [V]} \cdot 4095}{3,3 \text{ [V]}} = 1240,9$$

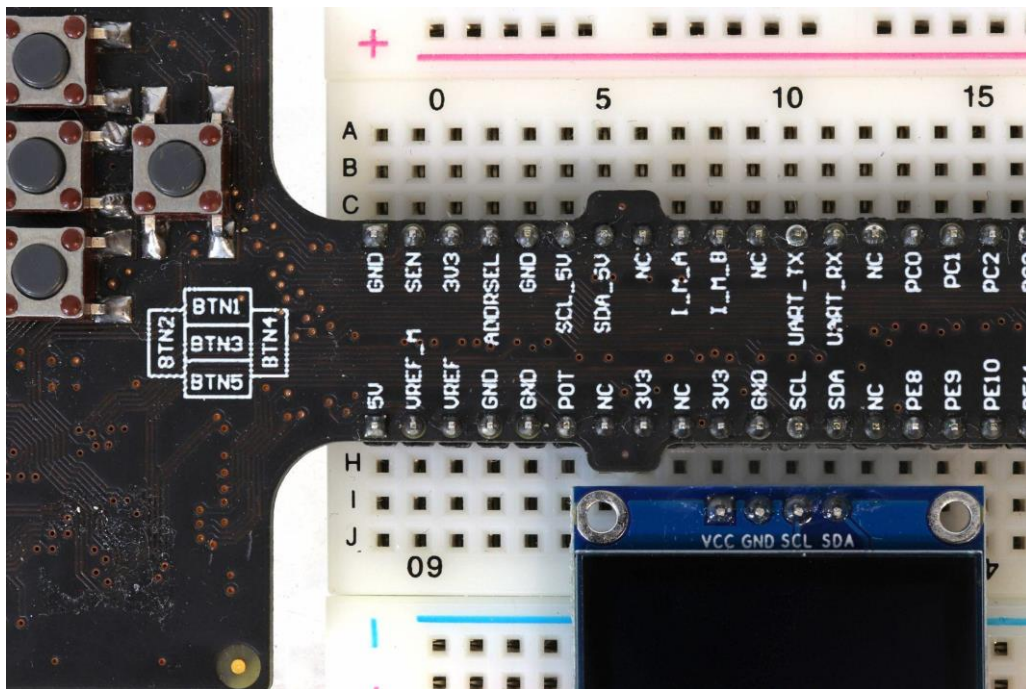
Mivel a regiszterbe egész számokat írhatunk, ezért 1 V kimenethez az 1241 a legközelebbi egész érték, így ezt kell a DHR -be tölteni. A következő bekezdésben ezt ki is fogjuk próbálni, valamint egy fix frekvenciájú szinuszelet is elő fogunk állítani a DAC segítségével.

DAC HASZNÁLATA A FEJLESZTŐKÁRTYÁN

A mikrovezérlő PA4 és PA5 lábaira csatlakoztatható a beépített DAC perifériára. A fejlesztőkártya úgy lett kialakítva, hogy a PA5 a PWM vezérlőjelet, a PA4 pedig a V_{REF} feszültséget állítja elő a DC motor számára.

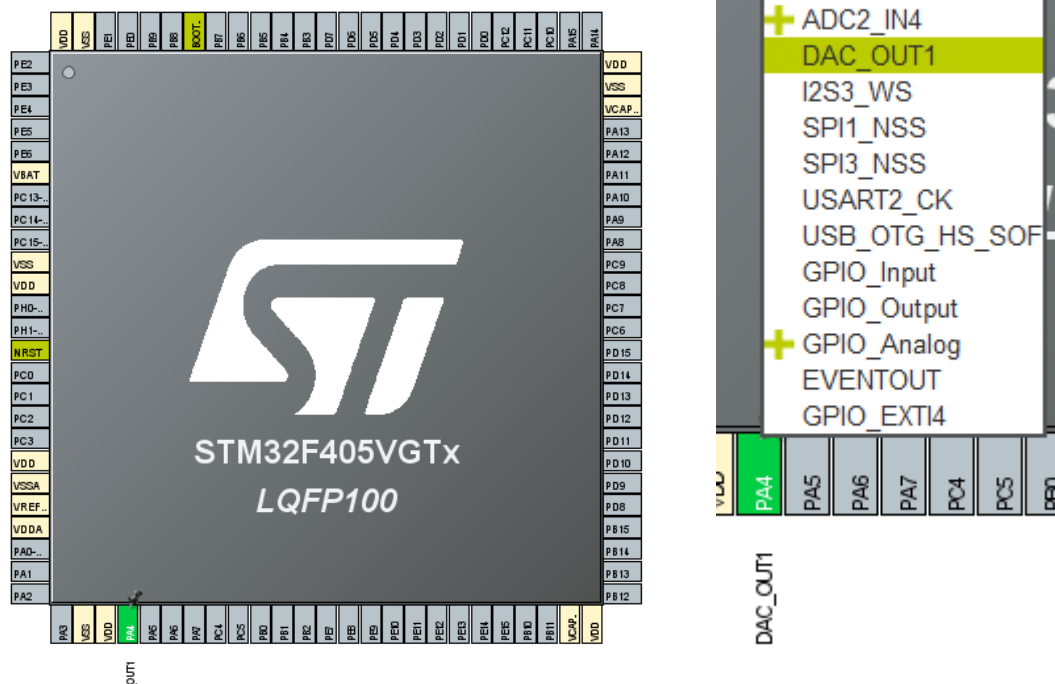
1. FELADAT: FIX KIMENŐFESZÜLTSG ELŐÁLLÍTÁSA

A feladat során a DAC-vel előállított jelet fogjuk egy multiméter segítségével a J2-es csatlakozó 2-es és 4-es lába között megmérni. Ezek a tűkesor V_{REF_M} és GND lábai, ami a tűkesor bal oldalán a második és a negyedik láb.



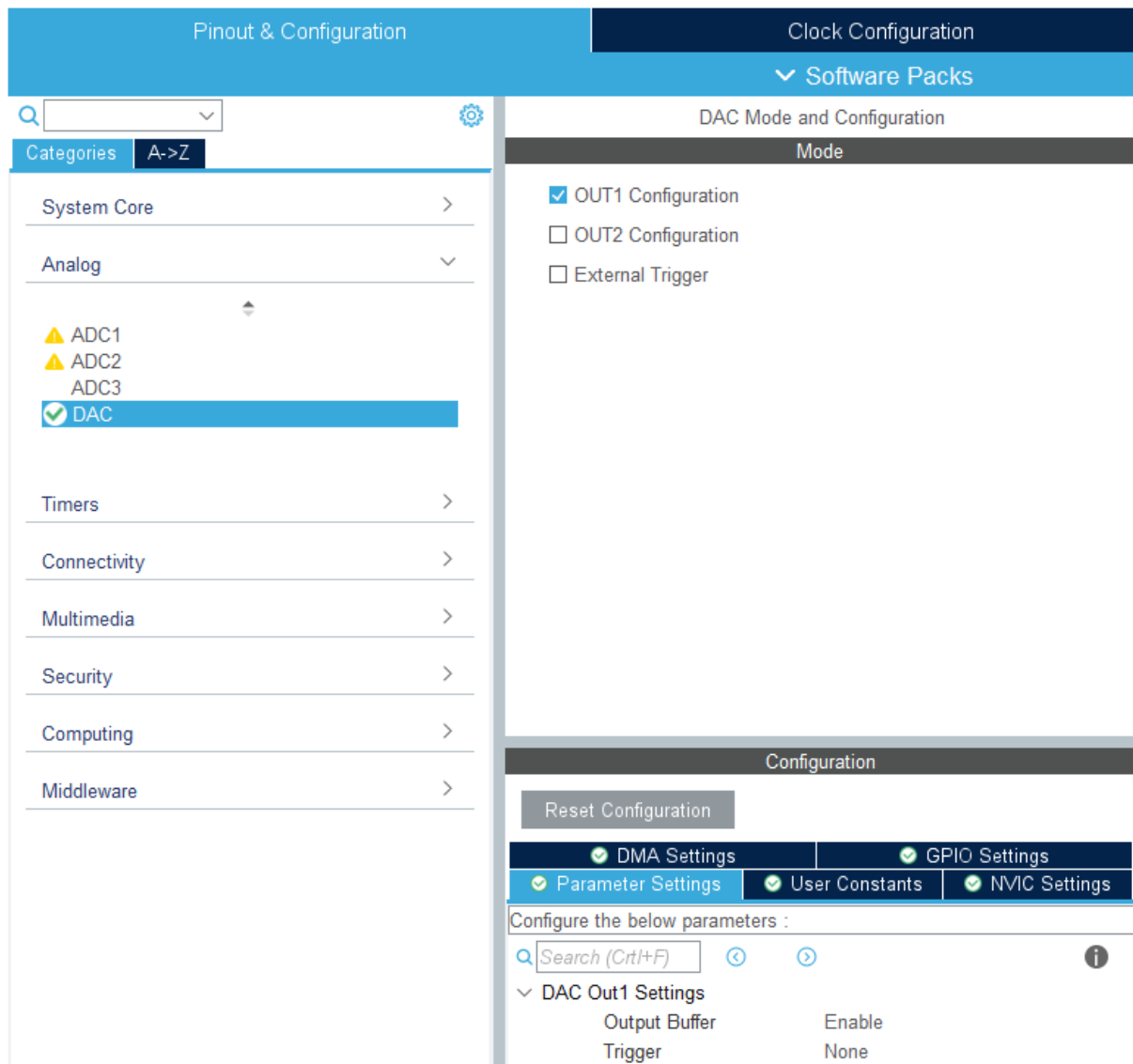
9. ábra: DAC által előállított feszültséghez tartozó lábak

A DAC által előállított jelet egy műveleti erősítő segítségével erősítjük, tehát az a DHR képlet eredményét 1,5-tel még el kell osztanunk (mivel a kimenet 5 V-os, de a mikrovezérlő 3,3 V-os feszültség szinten üzemel). Az STM32CubeIDE elindítása után a 4. tananyagrészen megismertek alapján állítsuk be a DAC modult majd a PA4 lábat DAC kimenetre. Ehhez a projekt megnyitása után a **Project Explorer**-ben kattintsunk az. ioc fájlra, ami a konfigurációkat tartalmazza. A megnyíló grafikus felületen könnyen beállítható a PA4 lábba (alsó sorban balról a negyedik láb) a **DAC_OUT1** kimenet.



10. ábra: DAC bekattintása a CubeMX felületen

A DAC-hez tartozó egyéb beállítások is a képen látható felületen állíthatók a legkönnyebben. A **Categories** fülön az **Analog** szekcióban a **DAC**-re kattintva láthatjuk az alap konfigurációt.



11. ábra: DAC beállítása

Ezt egyelőre hagyjuk érintetlenül, de a következő feladatban még vissza fogunk ide térni. Az .ioc fájl mentése közben a fejlesztőkörnyezet felajánlhatja, hogy automatikusan létrehozza a DAC vezérlő regiszterének beállításához szükséges struktúrát és függvényt, azonban, ha ezt nem teszi meg, akkor manuálisan kell elvégeztetnünk a kódgenerálást. Ezeket az elemeket a main.c-ben találjuk meg abban az esetben, ha nem pipáltuk ki azt a lehetőséget, hogy külön fájlokba generálja az adott perifériához tartozó függvényeket, valamint a saját programunkat is ide fogjuk írni. A main.c megnyitásához navigáljunk el a bal oldali **Project Explorer** menüben a **Core**, majd azon belül az **Src** könyvtárba! A main.c-ben a DAC kezeléséért felelős struktúra a következő:

```
// DAC struktúra létrehozása
DAC_HandleTypeDef hdac;
```

A beállításokat pedig a következő inicializáló függvény írja be a vezérlő regiszterbe:

```
/* DAC inicializáló függvény */
```

```
void MX_DAC_Init(void)
```

A legegyszerűbb implementáció esetén már csak néhány sor kód választ el minket az első sikeres digitális-analóg átalakításunktól. Keressük meg a main függvényt, majd a következő kommentet:

```
/* USER CODE BEGIN 2 */
```

Ennek a sornak a jelentése az, hogy itt kezdődik a felhasználói kód, és azért fontos, hogy a saját kódunkat ez után írjuk, mert ha módosítunk az .ioc fájlban és az új beállítások szerint újrageneráljuk a kódot, akkor a fejlesztőkörnyezet a nem felhasználói kód számára fenntartott helyen lévő kódrészleteket törölni fogja. Így ezen sor után írjuk a következőket.

```
/* Kimeneti érték megadása voltban_*/
```

```
float u_kimeneti = 1.0;
```

```
/*A digitális bemeneti érték kiszámítása, a műveleti erősítő figyelembevételével*/
```

```
uint32_t d_bemenet = (uint32_t)((u_kimeneti/1.5)*4095)/(3.3);
```

```
/* A digitális bemeneti érték beállítása */
```

```
HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, d_bemenet);
```

```
/* Az analóg-digitális átalakítás elindítása */
```

```
HAL_DAC_Start(&hdac, DAC_CHANNEL_1);
```

A kód fordítása és feltöltése után a VREF_M és a GND tűskék között 1 V feszültséget fogunk mérni.

2. FELADAT: SZINUSZJEL ELŐÁLLÍTÁSA

A feladat során a mikrovezérlővel kiszámoltatunk egy 100 pontból álló szinuszhullámot, majd ebből két módszerrel előállítjuk a kimeneti analóg jelet. A feladat során azt szeretnénk, hogy a szinuszjel frekvenciája 10 Hz legyen. Ehhez két megoldást is bemutatunk. Az első esetben a programunk fő ciklusát használva az előző feladattal megegyező módon pontról pontra állítjuk a kimenetet. Ebben az esetben a szinuszjel digitális értékeit tartalmazó tömbön egy ciklus segítségével haladunk végig. Mivel a szinuszjel egy periódusa 100 pontból áll, ezért másodpercenként tízszer kell a tömbön végighaladnunk, tehát másodpercenként ezerszer kell a kimeneti értéket megváltoztatnunk. Ezt úgy érjük el, hogy a kimenet beállítása után 1 ms késleltetést állítunk be. A **Private includes** részhez adjuk hozzá a math.h-t, amire a szinusz számítás miatt lesz szükségünk.

```

/* Private includes ----- */
/* USER CODE BEGIN Includes */
/* Matematikai könyvtár hozzáadása */
#include "math.h"
/* USER CODE END Includes */

```

A **USER CODE** résznél adjuk hozzá a szinuszhullám pontjainak kiszámítására szolgáló függvényt! A függvény bemenete az adatok tárolására szolgáló tömb pintere és hossza.

```

/* USER CODE BEGIN 0 */
// Szinusz generáló függvény
void GenerateSine (uint32_t* sine,uint32_t length)
{
    // Meghatározott mennyiségű pont generálása
    for (int i=0; i<length; i++)
    {
        // Egyes pontokhoz tartozó értékek számítása
        sine[i] = ((sin(i*2*M_PI/length) + 1)*(4096/2));
    }
}
/* USER CODE END 0 */

```

A következő lépésben írjuk be a következő sorokat a megfelelő helyre:

```

/* USER CODE BEGIN 2 */
...
/* a szinusz hullám pontjainak tárolására szolgáló tömb */
uint32_t sine_data[100];

/* A szinusz hullám pontjainak számítása */
GenerateSine(sine_data,100);
...
/* USER CODE END 2 */

```

Végül pedig a fő ciklusba írjuk be a következő sorokat az analóg kimenet beállításához:

```
/* Infinite loop */
/* USER CODE BEGIN WHILE */
while (1)
{
    for (int i = 0; i < 100; i++)
    {
        /* A digitális bemeneti érték beállítása */
        HAL_DAC_SetValue(&hdac, DAC_CHANNEL_1, DAC_ALIGN_12B_R, sine_data[i]);
        /* Az analóg-digitális átalakítás elindítása */
        HAL_DAC_Start(&hdac, DAC_CHANNEL_1);
        /* Késleltetés beállítása */
        HAL_Delay(1);
    }
}
/* USER CODE END WHILE */

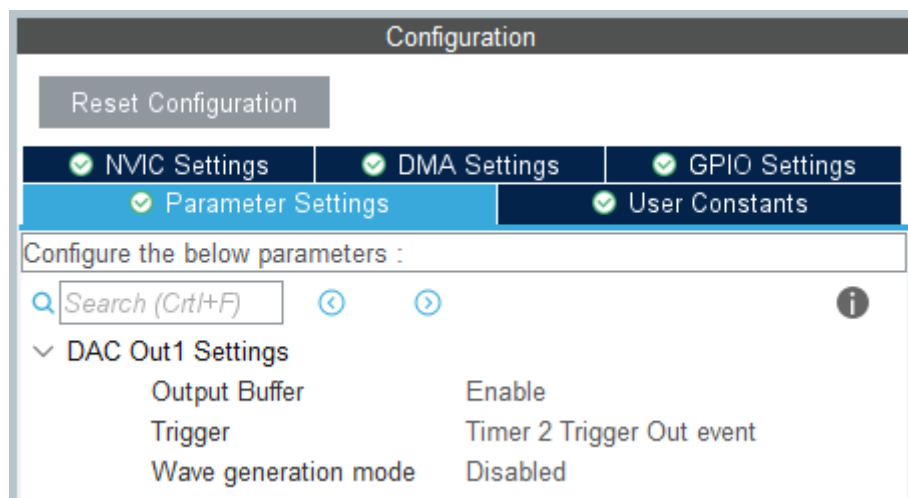
/* USER CODE BEGIN 3 */

/* USER CODE END 3 */
```

Ezután erre alkalmas eszközök (például oszcilloszkóp, multiméter vagy egy másik fejlesztőpanel megfelelő konfigurációban) segítségével ellenőrizhető a kimeneten a szinuszjel. Ebben az esetben a mikrovezérlőt semmi másra nem használjuk. Ez a megoldás a gyakorlatban nem alkalmazható olyan feladatban, ahol fontos, hogy az analóg jel frekvenciája fix legyen akkor is, ha új funkciókkal bővítjük a programunkat. Mi történik a szinuszjellel, ha másra is használjuk a mikrovezérlőnket? Ebben az esetben az 1 ms késleltetés már nem lenne helytálló, mert az adott feladat futási idejének függvényében kellene a késleltetést beállítani. Ez a gyakorlatban nem igazán megvalósítható, így a szinusz jel torzulni fog.

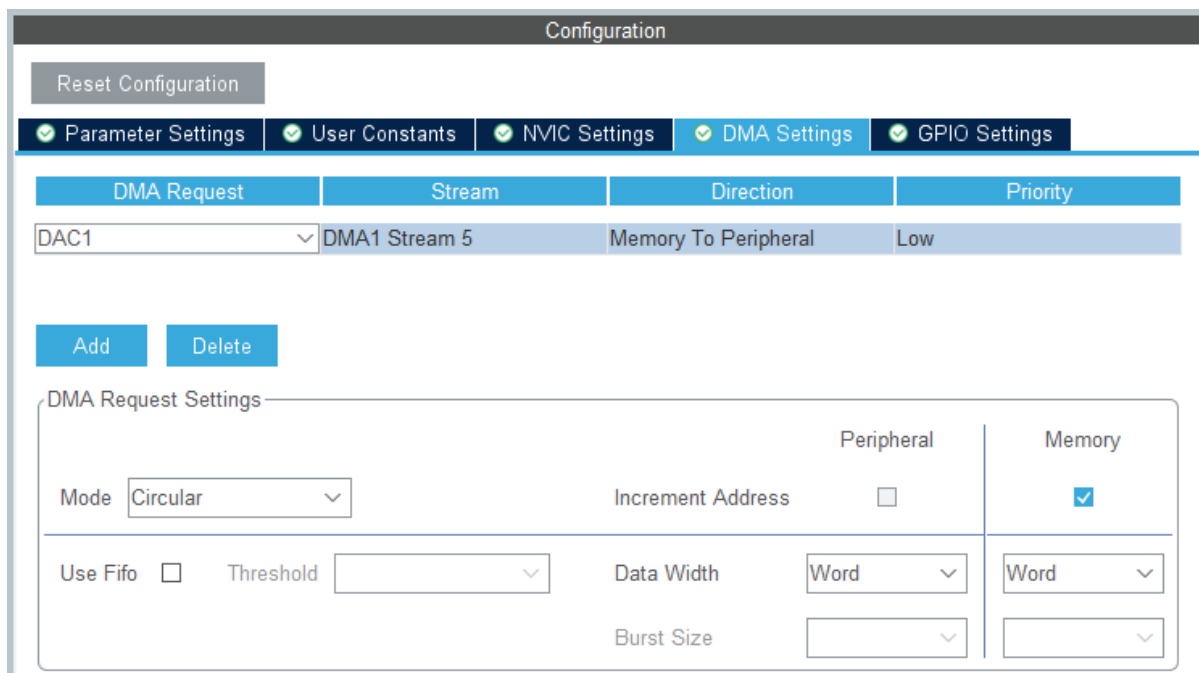
A következő megoldás arra ad példát, hogy mit tehetünk akkor, ha fontos, hogy a kimeneti jel frekvenciája pontos legyen. Például, ha zenét hallgatunk, miközben az interneten böngészünk, nem igazán örülnénk, ha oldalbetöltés közben a CPU terheltségétől függően lelassulna a zene. A mikrovezérlő perifériái beállíthatók úgy, hogy jelen feladatot beavatkozás és a CPU használata nélkül elvégezzék. Ha egy ilyen implementáció mellett később bővítjük a programunkat akár számításigényes feladatokkal is, annak semmilyen hatása nem lesz a szinuszjelünk frekvenciájára. Ehhez a nemrég említett DMA-t hívjuk segítségül. A DMA úgy működik, hogy egy adott periféria, jelen esetben a DAC

megkapja annak a memóriaterületnek a címét, ahol a szinusz pontjai találhatók. Ezután egy időzítő segítségével pontosan beállítható, hogy milyen gyakran frissüljön a kimenet. A kiszámolt pontokat tartalmazó tömb elemeit a Timer2-vel időzített DMA segítségével átadjuk a DAC perifériának. Lehetőség van még a kimeneti puffer letiltására és a beépített jelgenerátorok engedélyezésére is. A DMA-t körkörös módba állítjuk be, tehát ha az adattömb végére ér, akkor újra fogja kezdeni az elejétől. Amennyiben periodikus jeleket szeretnénk változás nélkül folyamatosan a kimenetre beállítani, akkor ezt a megoldást kell használni, de ha például zenét szeretnénk lejátszani, akkor a "Normal" módot kell választanunk, és a DMA lejártakor megszakítással (interrupttal) állítani a memóriát a hang következő szakaszára. A feladat megoldásához nyissuk meg újra az .ioc fájlt és keressük meg a DAC beállításait! Először állítsuk be a trigger feltételt a "Timer 2 Trigger Out event"-re. Ez azt jelenti, hogy az új értéket a kimenet a Timer 2 időzítő lejártakor fogja kapni.



12. ábra: DAC trigger feltétel beállítása

Ezt követően a **DMA Settings** fülön az **Add** gomb megnyomásával engedélyezzük a DMA-t és állítsuk be úgy, hogy a **Mode Circular** (körkörös) legyen és állítsuk át az adatszélességet ("Data Width") "Word"-re, hogy ez megegyezzen a szinuszhullám tárolásához használt tömb adatszélességével.

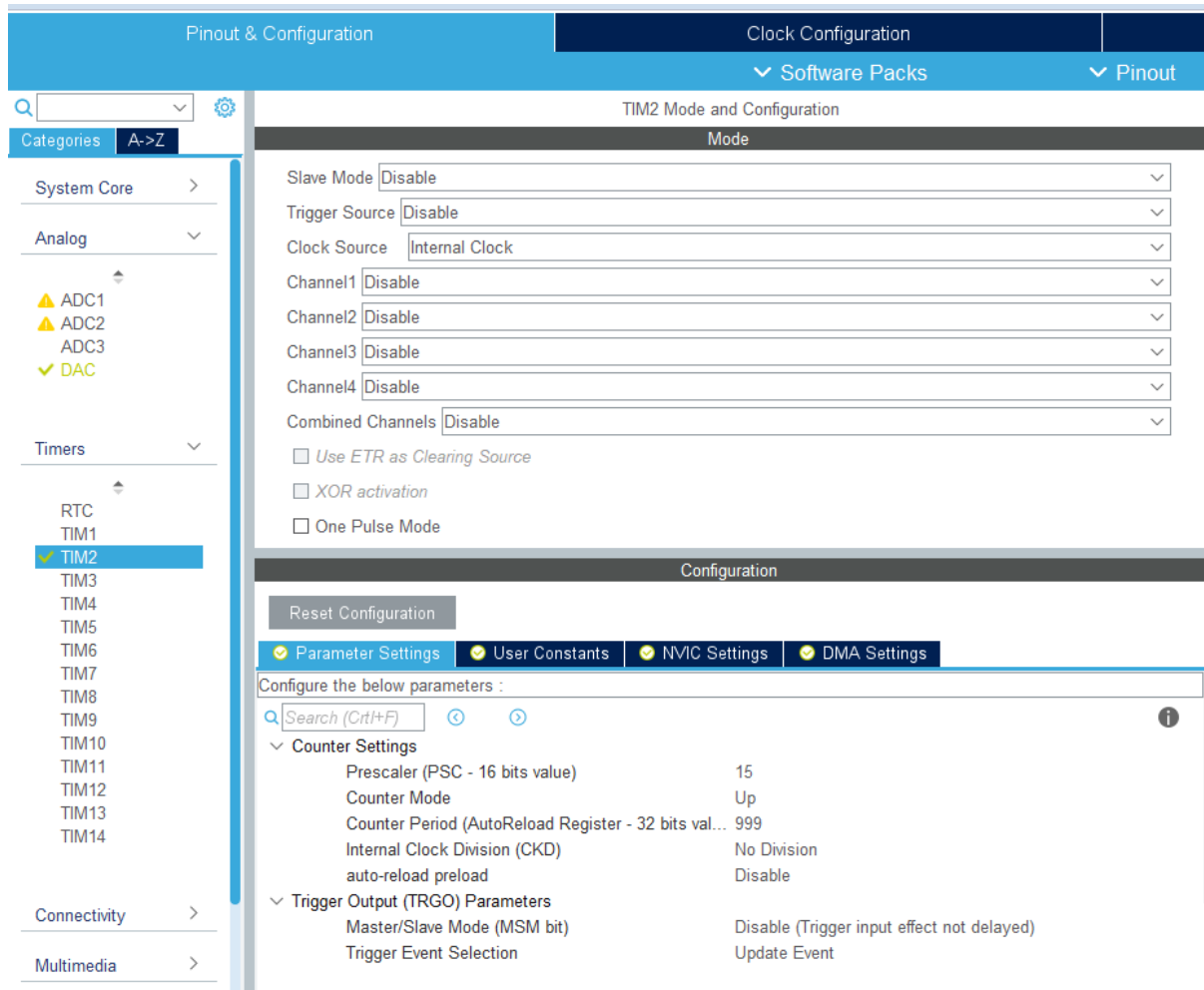


The screenshot shows the 'Configuration' window in STM32CubeMX. At the top, there's a 'Reset Configuration' button. Below it are tabs for 'Parameter Settings', 'User Constants', 'NVIC Settings', 'DMA Settings', and 'GPIO Settings'. The 'DMA Settings' tab is active. It displays a table with columns: 'DMA Request', 'Stream', 'Direction', and 'Priority'. The first row is configured for 'DAC1', 'DMA1 Stream 5', 'Memory To Peripheral', and 'Low'. Below the table are 'Add' and 'Delete' buttons. A 'DMA Request Settings' section is expanded, showing 'Mode' set to 'Circular', 'Increment Address' unchecked, 'Peripheral' checked, 'Use Fifo' unchecked, 'Threshold' set to a default value, 'Data Width' set to 'Word', and 'Burst Size' set to a default value.

13. ábra: DAC-hoz tartozó DMA beállítása

Ahhoz, hogy a szinuszel 10 Hz-es legyen, az időzítőt úgy kell beállítani, hogy másodpercenként ezerszer keletkezzen TRGO (Trigger output, kimeneti trigger) esemény, (ha a számláló eléri a megadott értéket felfelé számlálás, vagy a nulla értéket lefelé számlálás esetén) ami hatására a DAC a tömb következő elemét állítja a digitális bemenetre. Ehhez a Timer2 megszakításainak frekvenciáját 1 000 Hz-re kell választanunk. Ezt úgy tehetjük meg, hogy a **Clock Source**-ot állítsuk át **Internal Clock**-ra, ami azt jelenti, hogy a belső 16 MHz-es órajelről fog járni az időzítő. Majd a **Prescaler** (előosztó) segítségével elosztjuk az órajelet 1 MHz-re, ehhez 16-os osztásra van szükségünk. Mivel az osztás nullától való számlálással történik, ezért a Prescalert 15-re kell állítanunk. Ezután a **Counter Mode**-ot (számlálási mód) állítsuk **Up**-ra (fel, a másik opció a Down, azaz le), a **Counter Period**-ba pedig állítsunk be egyet kisebb számot, mint amit szeretnénk, jelen esetben 999-et, így másodpercenként 1 000 TRGO esemény fog keletkezni. Az Internal Clock Division (belső órajelosztó) beállításával az órajel további osztására van lehetőségünk, jelen esetben ezt most nem használjuk. Az auto-reload preload regisztert jelenleg tiltjuk le (Disable), engedélyezésére akkor van szükség, ha gyakran változtatjuk a programon belülről a Counter Period-ot és fontos, hogy rövid periódusok esetén is pontosak legyenek az átmenetek. A Trigger Output (TRGO) Master/Slave mód használatára több számláló összefűzésénél van szükségünk, jelenlegi feladatban legyen letiltva (Disable). A Trigger Event Selection-t (Trigger esemény választó) pedig állítsuk Update Event-re (frissítés esemény).

Az összes paraméter beállítása a mellékelt képen látható.



14. ábra: Időzítő beállítása

Ezután mentjük el az .ioc fájlt és módosítuk a programunkat az alábbiak szerint:

```
/* USER CODE BEGIN 2 */

/* a szinuszhullám pontjainak tárolására szolgáló tömb */
uint32_t sine_data[100];

/* A szinuszhullám pontjainak számítása */
GenerateSine(sine_data,100);

/* A DAC elindítása DMA támogatással */
HAL_DAC_Start_DMA(&hdac, DAC1_CHANNEL_1, sine_data, 100, DAC_ALIGN_12B_R);

/* A timer2 időzítő indítása */
HAL_TIM_Base_Start(&htim2);

/* USER CODE END 2 */
```



Valamint a fő ciklusból töröljük ki a kimenet kézi állítását, csak az üres ciklus maradjon:

```
/* Infinite loop */  
/* USER CODE BEGIN WHILE */  
while (1)  
{  
    /* USER CODE END WHILE */  
  
    /* USER CODE BEGIN 3 */  
}  
/* USER CODE END 3 */
```

A program lefordítása és feltöltése után a VREF_M és a GND lábak között egy 10 Hz-es szinuszjelet mérhetünk. A jel frekvenciájára nem lesz hatással, ha a későbbiekben a főciklusban megírjuk a saját programunkat. Ez annak köszönhető, hogy a DMA tehermentesíti a processzorunkat a gyakori adatmozgatás alól, így az fontosabb, nagyobb számítási teljesítményt igénylő műveleteket tud végezni.

ÖSSZEFOGLALÁS

A tananyagrészt során megismerkedtünk a digitális-analóg átalakítás jelentőségével és tipikus felhasználási területeivel. Áttekintettük a DAC átalakítók főbb jellemzőit és típusait, valamint a fejlesztőkártya segítségével a gyakorlatban is áttekintettük a DAC átalakítás alapjait. A tananyagrészt célja az volt, hogy egy olyan alapot adjon az olvasónak, amire építközhet, ha önállóan szeretne a témában elmélyülni.